

编译原理上机报告

《DBMS 的设计与实现》

学号： 23009201247 姓名： 郑墨涵

手机： 邮箱：

完成时间： 2026 年 5 月 26 日

目 录

1 项目概况	3
1.1 基本目标	3
1.2 完成情况	3
2 项目实现方案	7
2.1 技术路线与架构	7
2.2 逻辑结构与物理结构	10
2.3 语法结构与数据结构	14
2.4 执行流程	18
2.5 功能测试	23
3 总结与未来工作	31
3.1 未完成功能	31
3.2 未来实现方案	32

1 项目概况

1.1 基本目标

本项目的基本目标是设计并实现一个关系型数据库管理系统 (DBMS) 的原型系统。系统需能够接收基本的标准 SQL 语句, 对其进行词法分析、语法分析, 并将其转化为一棵抽象语法树 (AST)。最后, 通过底层的语义执行引擎解释执行这棵语法树, 完成对底层数据库物理文件的读写操作。项目旨在将《编译原理》中的前端技术 (正规式、Lex、上下文无关文法、Yacc、语法制导翻译) 与《数据库系统》中的底层存储技术紧密结合。

1.2 完成情况

1.2.1 采用的方案与技术栈

本项目在技术选型上进行了创新, 摒弃了传统且容易引发内存管理风险的 C/C++ 与 Lex/Yacc 工具链, 转而采用敏捷高效的 Python 语言及其编译原理工具库 PLY (Python Lex-Yacc) 进行核心模块的开发。在系统的全生命周期设计与编码过程中, 深度引入了大语言模型 (LLM, Gemini 1.5 Pro) 作为 AI 辅助编程与逻辑校验工具, 大幅提升了开发效率与代码健壮性。

1.2.2 LLM 辅助的 DBMS 设计与实现细节

在本项目中, LLM 在以下五个关键阶段发挥了核心辅助作用:

- **(1) 词法分析器构建:** 利用 LLM 辅助生成了 SQL 关键字的正则表达式规则, 并采纳其建议实现了“查表法” (构建 reserved 字典) 来替代繁琐

的正则枚举，优雅地解决了 SQL 关键字大小写不敏感的问题；同时借助 LLM 修复了新版 Python 引擎下 PLY 库关于全局正则 Flag 的兼容性报错。

- **(2) 语法分析器构建：**通过向 LLM 提供 SQL 的 BNF 文法草图，辅助自动生成了对应的 PLY Yacc 解析规则；在处理 WHERE 子句中 AND 与 OR 混合使用的复杂场景时，利用 LLM 辅助定义了算符优先级（precedence），成功消除了移进-归约冲突（Shift-Reduce Conflict）；同时自动生成了轻量级的 Python 字典嵌套结构作为 AST（抽象语法树）节点。

- **(3) 语法制导翻译：**LLM 辅助生成了自底向上的语义动作模板，自动完成了从产生式右部提取属性（如 \$1, \$3）并向左部（\$\$）传递的 AST 构造代码。

- **(4) 存储逻辑结构设计：**利用 LLM 自动生成了针对 sys.dat 元数据文件的读写与解析代码，实现了从建表语句到数据字典映射的序列化操作。

- **(5) 存储物理结构设计：**LLM 辅助生成了基本表 .txt 文件的增量追加（Insert）与全量覆写（Update/Delete）的 I/O 模块模板；特别是在实现多表联查功能时，借助 LLM 提供的 itertools.product 算法，成功实现了底层数据行的笛卡尔积运算。

1.2.3 已经实现的功能与 SQL 语句

目前系统已成功实现并支持交互式终端（REPL）执行的语句包括：

- **数据库定义 (DDL):**
 - CREATE DATABASE <库名>; （创建数据库及专属物理目录）
 - SHOW DATABASES; （查看系统当前所有数据库）

- USE <库名>; (切换当前工作数据库环境)
- DROP DATABASE <库名>; (级联删除数据库及其下属所有表文件)
- CREATE TABLE <表名> (列名 类型, ...); (支持 INT 和 CHAR 类型, 具备同名表防重校验)
- SHOW TABLES; (显示当前数据库下的所有表)
- DROP TABLE <表名>; (删除表数据及清理数据字典)
- **数据操纵 (DML):**
 - INSERT INTO <表名> VALUES (...); (全量字段数据插入)
 - INSERT INTO <表名> (列 1, 列 2) VALUES (...); (**扩展:** 指定列插入, 未指定列自动补全为 NULL)
 - UPDATE <表名> SET 列 1=值 1, 列 2=值 2 WHERE <条件>; (**扩展:** 支持多字段同时更新)
 - DELETE FROM <表名> WHERE <条件>; (条件删除元组)
- **数据查询 (DQL) 与复合逻辑条件:**
 - SELECT * FROM <表名>; (全表查询)
 - SELECT 列 1, 列 2 FROM <表名> WHERE <条件>; (指定列投影与条件选择)
 - SELECT * FROM <表 1>, <表 2> WHERE <条件>; (**扩展:** 支持多表联合查询与笛卡尔积生成)
 - 复合 WHERE 条件: 底层依托递归 AST 树, 完美支持 >=, <=, != 等比较运算, 以及带有嵌套括号 () 和 AND、OR 逻辑组合的复杂条件判定。

2 项目实施方案

2.1 技术路线与架构

本 DBMS 原型系统采用了“经典编译原理前端 + 纯文本文件存储后端 + LLM 深度辅助”的技术路线。系统以 Python 语言为核心开发环境，采用纯 Python 实现的 PLY (Python Lex-Yacc) 库，极大提升了开发效率与内存安全性，同时深度整合了大语言模型以加速各模块的构建与查漏补缺。

2.1.1 系统模块划分

系统整体架构分为四大核心模块：

1. **词法分析器 (Lexer)**: 基于 `ply.lex` 实现。采用正则表达式定义 Token 匹配规则，并通过“查表法”（预定义 `reserved` 字典）高效区分关键字与普通标识符 (ID)，实现了 SQL 语言的大小写不敏感特性。
2. **语法分析器 (Parser)**: 基于 `ply.yacc` 实现。采用 LALR(1) 语法分析算法，将 SQL 语句的 BNF 产生式规则映射为 Python 函数。在移进-归约过程中，利用算符优先关系 (`precedence`) 解决二义性冲突。
3. **抽象语法树 (AST)**: 抛弃了传统 C 语言复杂的 `struct` 链表指针，采用 Python 原生的字典 (`dict`) 与列表 (`list`) 嵌套结构，自底向上构建轻量级的抽象语法树，用于保存表名、字段定义、WHERE 复合条件（嵌套逻辑树）等语义信息。
4. **底层存储引擎 (Storage Engine)**: 采用直接的文件 I/O 操作。元数据 (Schema) 集中存储于 `sys.dat` 数据字典中；用户基本表数据采用纯

文本格式，以 表名.txt 的形式独立存储。通过 Python 的 `itertools.product` 实现多表查询时的笛卡尔积运算。

2.1.2 大语言模型 (LLM) 信息

在项目开发过程中，大量借助了 LLM 辅助代码生成与逻辑验证：

- 模型名称：Gemini
- 模型版本：Gemini 1.5 Pro
- 提供商：Google
- 模型参数量/微调概况：千亿级参数大模型，采用 Zero-shot 与 Few-shot Prompting 技术。利用其在代码生成（尤其是编译原理底层算法库的调用）领域的泛化能力，直接对接到 PLY 库的开发需求。

2.1.3 Prompt 规范与应用示例

针对不同的开发任务，设计了结构化的 Prompt 模板，以确保 LLM 输出代码的可用性与规范性。

- 关键任务 1：自动生成 Lex 正则表达式规则
 - **Prompt 模板：**你是一个编译原理专家。请使用 Python 的 PLY 库，为以下 SQL 关键字和符号编写对应的词法规则代码。要求：忽略大小写，且必须采用查字典的方式处理关键字以避免正则冲突。待处理关键字：{SQL_KEYWORDS}
 - **输入示例：**待处理关键字：CREATE, TABLE, SELECT, WHERE
 - **输出示例：**生成包含 reserved 字典定义及 `def t_ID(t):`

`t.type = reserved.get(t.value.lower(), 'ID');` return `t` 的高质量 Python 代码。

- **关键任务 2：自动生成 Yacc 语法规则与 AST 构造**

- **Prompt 模板：**请根据以下 SQL 语句的 BNF 文法，使用 PLY 的 yacc 编写解析规则函数，并在归约时使用 Python 字典构造对应的 AST。文法：{BNF_GRAMMAR}

- **输入示例：**文法：`condition : condition AND condition | ID op value`

- **输出示例：**生成 `def p_condition_logic(p): p[0] = {'type': 'logic', 'op': 'AND', 'left': p[1], 'right': p[3]}` 等代码，并自动添加 precedence 优先级定义以消除 Shift/Reduce 冲突。

2.1.4 LLM 辅助设计点详述

LLM 深度参与了系统从前端到后端的多个设计阶段：

1. **词法分析器层面：**自动生成复杂符号（如单引号包裹的字符串 `t_STRING = r"'[^']*'"`）的正则表达式；自动补充边界测试样例（如包含空格和混合大小写的异形 SQL 语句）。

2. **语法分析器层面：**辅助编写 LALR(1) 文法的消除左递归与提取左公因子策略；自动发现并解决 AND 和 OR 混合使用时的语法冲突问题；自动生成 AST 节点的 JSON/字典结构定义；生成详尽的语法错误提示语句（如捕获非法分号位置）。

3. **语法制导翻译层面：**自动生成 `execute_create`、`execute_select` 等语义动作模板；辅助完成嵌套括号运算的 `check_condition` 递归执行函数的

编写。

4. **存储逻辑与物理层面**：自动生成 `sys.dat` 元数据解析代码的模板（如按空格切分、提取表名与列长）；生成多表联查时的笛卡尔积 (`itertools.product`) 核心算法；生成文件增量追加写入 ('a' 模式) 和覆写 ('w' 模式) 的安全 I/O 模块模板。

2.1.5 回退策略

由于 LLM 生成的编译器代码存在逻辑漏洞或版本不兼容（如 PLY 对 `docstring` 格式的严格要求）的风险，系统制定了以下异常回退策略：

1. **格式不合法回退**：当 LLM 生成的 Yacc 产生式中包含同行多个 `|` 导致 PLY 抛出 `Illegal name '|' in rule` 异常时，通过向 LLM 反馈错误 `Traceback` 进行二次迭代纠正，强制其重构为换行书写的规范格式。
2. **逻辑语义回退**：若 LLM 生成的执行器在多表查询时引发内存溢出或逻辑错误（如笛卡尔积匹配失败），则放弃 LLM 方案，回退至人工手写的简单嵌套 `for` 循环过滤机制。
3. **AST 结构回退**：如果模型生成的 AST 嵌套过深难以解析，则向其提供预期 AST 的标准字典快照（JSON 格式），利用 `Few-shot Prompting` 约束其输出结构，使其严格对齐预期格式。

2.2 逻辑结构与物理结构

本系统的底层存储引擎遵循了“元数据（Metadata）与用户数据（User Data）分离”的经典数据库设计原则。考虑到本系统为一用于编译原理实践的轻量级 DBMS 原型，为了降低文件管理的复杂度并提高调试的直观性，底层并未采用

传统商业数据库复杂的页（Page）、块（Block）以及段（Segment）的二进制存储架构，而是独创性地设计了一套基于操作系统文件系统的“目录-文本文件”映射体系来进行持久化存储。

2.2.1 元数据的逻辑结构

为了支持 CREATE DATABASE、USE 以及 CREATE TABLE 等多数据库、多表管理功能，系统引入了两级元数据结构：

1. **全局元数据**：存储在系统根目录下，仅记录当前 DBMS 中创建了哪些数据库（对应各个数据库文件夹的名称）。
2. **数据库级元数据（数据字典/Catalog）**：存储在每个数据库特定的作用域内，用于详细记录该数据库下所有表的结构定义（Schema），即字段名、类型、长度及顺序。

字段名称	说明	类型示例
TableName	表名	CHAR (如 student)
ColIndex	列序号	INT (如 1, 2, 3)
ColName	列名	CHAR (如 Sno, Sname)
ColType	列数据类型	CHAR (如 int, char)
ColLength	数据长度	INT (如 9, 20, 4)

表 1 数据库级元数据（数据字典）的逻辑结构

2.2.2 物理结构与图形说明

在物理磁盘层面，系统将操作系统的目录树（Directory Tree）映射为数据库的层级。具体映射规则如下：

- **数据库** 物理化为：根目录下的一个文件夹。
- **元数据** 物理化为：名为 `sys.dat` 的纯文本文件。
- **基本表数据** 物理化为：数据库文件夹下名为 表名.txt 的纯文本文件，每一行代表一条元组（Record），属性值之间以空格分隔。

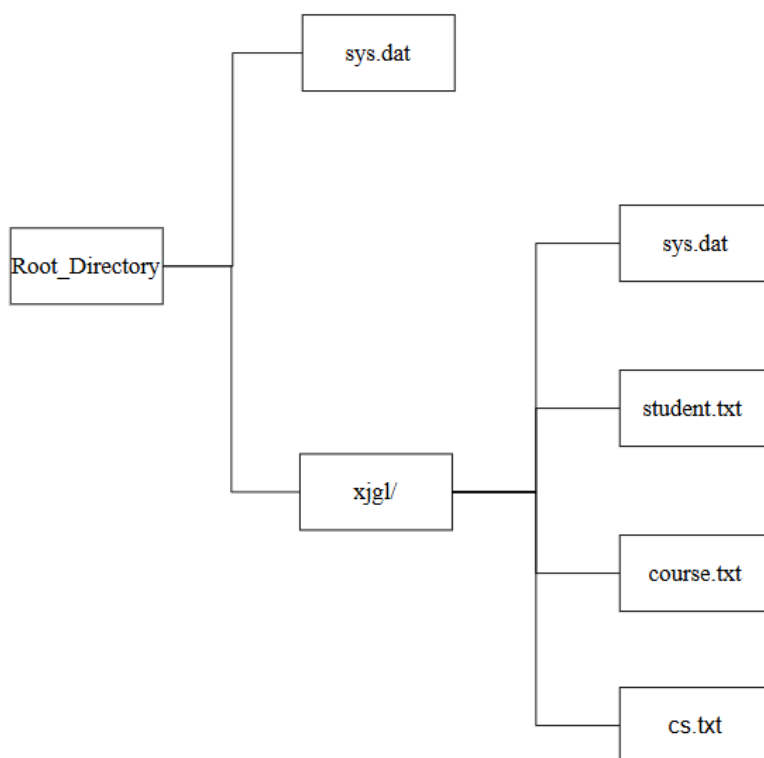


图 1 DBMS 物理存储结构示意图

2.2.3 数据保存实例

以在 `xjgl` 数据库中执行 `CREATE TABLE student (sno CHAR(9), sname CHAR(20), sage INT)`; 并插入两条数据为例，底层物理文件的真实存储状态如下：

局部 `xjgl/sys.dat` (数据字典文件) 实例：

student 1 sno char 9

student 2 sname char 20

student 3 sage int 4

说明：上述每一行准确映射了“表名 序号 列名 类型 长度”的元数据逻辑结构。

数据文件 xjgl/student.txt (基本表文件) 实例：

01 ZHANGSAN 20

02 LISI 22

说明：数据按行存储，当发生 DELETE 操作时，系统会将满足条件的行在内存中过滤，随后覆写回该 student.txt 文件；如果发生部分列的 INSERT，未指定的列将被自动补全为 NULL 字符串写入此文件。

2.2.4 物理结构的优缺点分析

结合操作系统的底层机制，对上述“目录-文本文件”物理结构的分析如下：

优点：

1. **极高的开发敏捷性与可解释性：**纯文本存储完全透明（白盒），在开发与期末验收调试时，可以直接使用操作系统的文本编辑器（如 Notepad）打开 sys.dat 和 .txt 文件，实时核对 SQL 执行后的物理落盘情况，对理解“数据是如何持久化到硬盘的”这一核心概念具有极佳的教育意义。

2. **跨平台兼容性强：**完全依赖 Python 标准库（如 os、shutil）的原生文件流（File Stream）进行 I/O 操作，避开了特定操作系统的扇区（Sector）和簇（Cluster）管理机制，使该 DBMS 可以无缝运行在 Windows、Linux 和 macOS 上。

缺点：

1. **I/O 效率遭遇瓶颈**：由于物理文件没有按 4KB 或 8KB 划分页（Page）或槽（Slot），系统无法建立内存缓冲池（Buffer Pool）。每一次 SELECT、UPDATE 或 DELETE，都必须将整个 .txt 文件按行遍历全部加载到内存（即强迫进行全表顺序扫描 Sequential Scan），当数据量达到十万级时，磁盘 I/O 开销将呈线性剧增。

2. **缺乏物理索引机制的支持**：文本流按行存储的特性导致我们无法计算出某一条元组固定的物理偏移地址（Offset），因此无法在此基础上构建 B+ 树或 Hash 散列等高效索引结构，致使复合条件查询（如 WHERE sage = 20）的时间复杂度始终为 $O(N)$ 。

3. **并发安全与崩溃恢复能力的缺失**：当前的纯文本覆写机制（'w' 模式）缺乏文件级别的读写锁（如 S 锁/X 锁）控制。若在执行 UPDATE 全量覆写物理文件期间发生进程崩溃或意外断电，极大概率会导致原 .txt 文件被清空且无法恢复（因为系统未设计类似 WAL 的预写式重做日志机制）。

2.3 语法结构与数据结构

在传统 C 语言的 Yacc 实现中，非终结符的综合属性（即 \$\$）通常需要通过定义极其复杂的 struct 结构体和大量的指针链表（如 next_fdef）来保存，这不仅容易引发内存泄漏，而且开发效率极低。

在本次 Python + PLY 的实现中，我摒弃了 C 语言的指针结构，转而利用 Python 原生的字典(Dictionary) 和列表(List) 作为数据结构来承载非终结符的属性。这种方式不仅天然支持无限深度的层级嵌套，还使得自底向上构建出的抽象语法树 (AST) 以极度直观的 JSON/字典形式呈现。

下面逐一详细说明核心扩展 SQL 语句的语法结构（产生式）与对应的数据结构。

1. CREATE TABLE 语句

产生式语法结构：

`createsql : CREATE TABLE ID LPAREN fieldsdefinition RPAREN SEMI`

`fieldsdefinition : field_type | fieldsdefinition COMMA field_type`

`field_type : ID CHAR LPAREN NUMBER RPAREN | ID INT`

数据结构说明：

非终结符 `createsql` 最终归约生成一个根节点字典。该字典包含两个键：

- `table_name`: 字符串，保存目标表的名称。
- `fields`: 列表，保存该表所有的字段定义。列表中的每一个元素也是一个字典，包含 `name` (列名)、`type` (类型) 和 `length` (占用的字节长度)。

实例说明：

输入语句：`CREATE TABLE Student (Sno CHAR(9), Sage INT);`

其在内存中自底向上归约后，对应的数据结构如下：

```
{
  "table_name": "Student",
  "fields": [
    {"name": "Sno", "type": "char", "length": 9},
    {"name": "Sage", "type": "int", "length": 4}
  ]
}
```

2. 带有复合条件的 SELECT 语句（重点难点）

产生式语法结构:

selectsql : SELECT select_fields FROM tables_list WHERE condition SEMI

condition : condition AND condition

| condition OR condition

| LPAREN condition RPAREN

| ID op value

数据结构说明:

为了支持无限层级的括号嵌套以及 AND、OR 的逻辑优先级, 非终结符 condition 被设计为一棵递归逻辑树结构。它会生成两种类型的节点字典:

1. 基础比较节点 (type: 'cmp'): 叶子节点, 包含 col (参与比较的列)、op (操作符, 如 GT/EQ) 和 val (目标值)。

2. 逻辑组合节点 (type: 'logic'): 分支节点, 包含 op (逻辑运算符 AND/OR)、left (左子树指针) 和 right (右子树指针)。

实例说明:

输入语句: SELECT Sname, Sage FROM Student WHERE (Sage > 20) AND (Ssex = 0);

其在内存中归约后, 对应生成的 AST 数据结构如下:

```
{  
  "fields": ["Sname", "Sage"],  
  "tables": ["Student"],  
  "where": {  
    "type": "logic",  
    "op": "AND",
```



```

    "left": {"type": "cmp", "col": "Sage", "op": "GT", "val": 20},
    "right": {"type": "cmp", "col": "Ssex", "op": "EQ", "val": 0}
  }
}

```

3. 带有指定列的多字段 UPDATE 语句

产生式语法结构:

updatesql : UPDATE ID SET set_list WHERE condition SEMI

set_list : set_item | set_list COMMA set_item

set_item : ID EQ value

数据结构说明:

非终结符 updatesql 包含三个核心属性: 目标表 table_name, 由 set_item 组成的修改列表 set_list, 以及过滤条件 where 树。

实例说明:

输入语句: UPDATE Student SET Sage = 27, Ssex = 1 WHERE Sname = 'ZHANGSAN';

对应生成的数据结构如下:

```

{
  "table_name": "Student",
  "set_list": [
    {"col": "Sage", "val": 27},
    {"col": "Ssex", "val": 1}
  ],
  "where": {"type": "cmp", "col": "Sname", "op": "EQ", "val":

```

```
"ZHANGSAN"}  
  
}
```

4. 抽象语法树 (AST) 数据结构图示

以 `SELECT Sname, Sage FROM Student WHERE (Sage > 20) AND (Ssex = 0);` 语句为例，其实际生成的完整抽象语法树 (AST) 数据结构图形化表示如下：

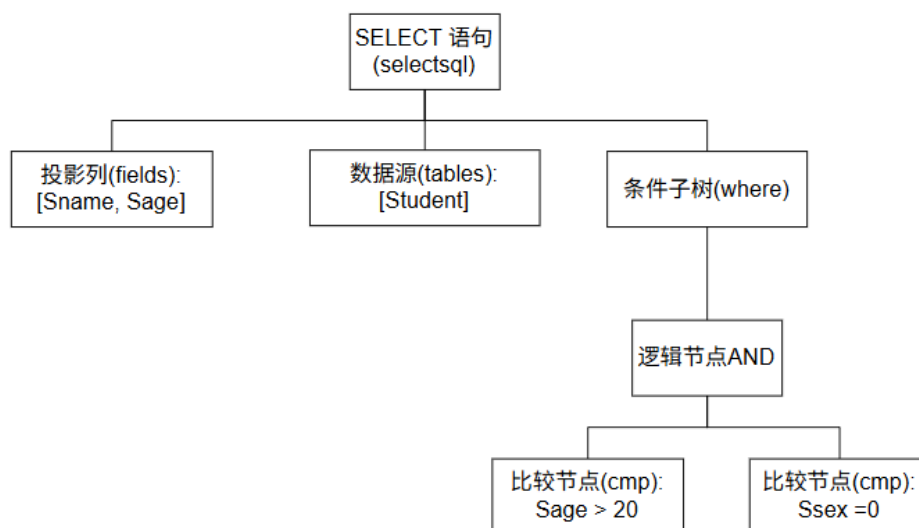


图 2 数据结构图

2.4 执行流程

本系统的执行引擎 (Execution Engine) 负责接收语法分析器 (Parser) 生成的抽象语法树 (AST 字典)，并将其映射为底层文件系统的 I/O 读写与关系代数运算。以下选取最核心的条件判断子程序，以及典型的建表、插入、查询流程进行详细说明。

2.4.1 核心语义子程序：条件鉴别与逻辑递归

为了支持 WHERE 子句中复杂的 AND/OR 嵌套逻辑，系统设计了专门的递归运算引擎。

函数名称: `check_condition`

函数说明: 结合给定的表结构 (Schema), 通过递归遍历 AST 条件逻辑树, 对读取到的物理文件数据行进行真值评估, 判断该元组是否满足用户输入的 WHERE 约束。

输入参数: *row: List 类型, 表示当前正在评估的一行具体数据 (字符串列表)。

- schema: List 类型, 当前遍历表的元数据字典集合。
- condition: Dict 类型, AST 中的 where 子树节点。

输出参数: Boolean 布尔值 (True 表示该行符合条件予以保留, False 表示不符合条件予以剔除)。

执行流程:

1. **判空跳出:** 若 condition 为空 (即用户未输入 WHERE 子句), 直接返回 True。

2. **逻辑分支处理 (Logic Node):** 若当前节点类型为 logic, 则递归调用 `check_condition` 分别计算其 left (左子树) 和 right (右子树) 的布尔值。最后根据当前节点的 op (AND 或 OR) 进行 Python 逻辑运算并返回合并结果。

3. **比较分支处理 (Cmp Node):** 若当前节点类型为 cmp, 则根据 col (目标列名) 在 schema 中查找对应的物理列索引; 提取 row 列表中该索引处的实际数据; 将其与 AST 中的目标 val 进行类型转换 (如比较大小时统一转换为 int); 执行 >, <, =, != 等比较操作并返回最终布尔值。

2.4.2 DDL 语句执行流程：创建关系表

函数名称： `execute_create_table`

函数说明： 解析建表的 AST 节点，更新数据字典，并在文件系统中初始化底层的物理数据文件。

输入参数： `ast`: Dict 类型，包含 `table_name`（目标表名）和 `fields`（字段定义列表）的语法树节点。

输出参数： 无返回值（直接向控制台输出执行成功或报错提示）。

执行流程：

1. **环境上下文检查：** 检查全局变量 `CURRENT_DB`，若为空则提示用户必须先执行 `USE DATABASE`。
2. **重名实体校验：** 调用 `get_schema(table_name)` 去局部的 `sys.dat` 中探查。若返回值不为空，说明该表已存在，抛出“表已存在”的异常并终止执行。
3. **元数据持久化：** 以追加模式（'a'）打开当前数据库的 `sys.dat`，遍历 `ast['fields']`，将新表的字段名称、类型、长度及排列序号按行写入字典文件。
4. **物理文件初始化：** 以覆写模式（'w'）在当前数据库目录下创建一个空的 `表名.txt` 文件，完成建表操作。

2.4.3 DML 语句执行流程：数据插入（支持残缺列映射）

函数名称： `execute_insert`

函数说明： 接收插入语句的 AST，将用户提供的数据序列化，并按行追加

到对应的物理文本文件中。支持全量字段插入与指定列插入（未指定列自动补全 NULL）。

输入参数： `ast`: Dict 类型，包含 `table_name`、`insert_fields`（用户显式指定的列，可为空）以及 `values`（待插入的数据值）。

输出参数： 无返回值。

执行流程：

1. **合法性前置检查：** 检查当前是否已选定数据库，并验证目标 表名.txt 文件是否存在。
2. **初始化空元组：** 调用 `get_schema` 获取该表的总列数 `$N$`，初始化一个长度为 `$N$`、元素全为 "NULL" 的字符串列表 `row_data_list`。
3. **数据映射填充：**
 - 若 `insert_fields` 为空（全量插入）：比对输入值数量与总列数是否一致。若一致，则直接按顺序将 `values` 覆写到 `row_data_list` 中。
 - 若 `insert_fields` 不为空（指定列插入）：比对指定列数与输入值数量。遍历用户指定的列名，在 `schema` 中查找其对应的绝对索引位置，将对应的值填入 `row_data_list` 的准确位置（其余位置保留 "NULL"）。
4. **物理落盘：** 将拼接好的 `row_data_list` 转换为以空格分隔的字符串，以追加模式写入 表名.txt 文件并换行。

2.4.4 DQL 语句执行流程：多表联合复杂查询

函数名称： `execute_select`

函数说明： 负责实现关系代数中的投影（Projection）、选择（Selection）以及基于多表的笛卡尔积（Cartesian Product）连接。

输入参数： `ast`: Dict 类型，包含 `fields`（投影列）、`tables`（数据源表列表）和 `where`（复合条件逻辑树）。

输出参数： 无返回值（直接向控制台格式化打印二维表格结果）。

执行流程：

1. **数据源收集：** 遍历 AST 中的 `tables` 列表，依次加载每一张表的 `schema` 和 `.txt` 物理文件中的所有文本行。

2. **Schema 拼接：** 将各张表的字段信息重新计算索引偏移量，组合为一个全局的 `combined_schema`（如果包含多表，会自动为列名添加 表名. 的前缀以防重名冲突）。

3. **关系代数连接(笛卡尔积计算)：** 利用 Python 的 `itertools.product` 算法，将加载进内存的多张表的数据行进行全排列组合，生成一张庞大的全属性虚拟宽表 `cartesian_rows`。

4. **投影列解析：** 解析 AST 中的 `fields` 列表（若是 `*` 则映射为 `combined_schema` 的全集），提取出用户最终需要展示的列索引和表头名称。

5. **条件过滤与格式化输出：** 遍历虚拟宽表 `cartesian_rows` 的每一行，将其连同 `combined_schema` 送入 `check_condition()` 函数进行真假鉴定。对于返回 `True` 的行，依据上一步提取的投影列索引切片截取数据，格式化对齐并打印输出。

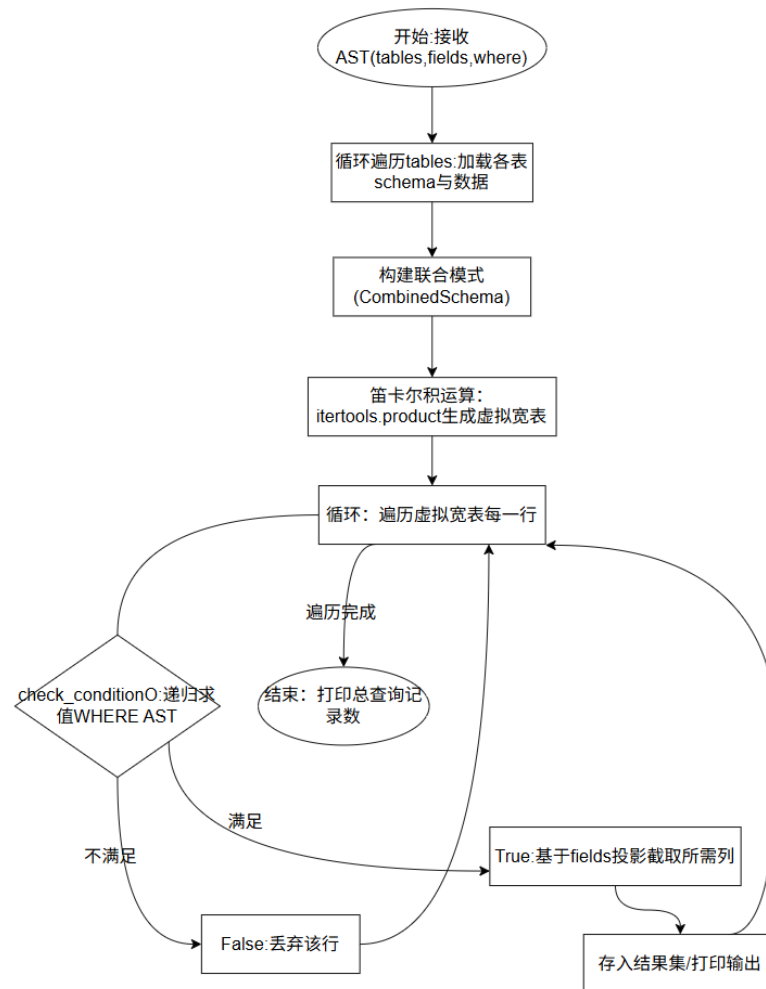


图 3 SELECT 多表联合查询执行流程图

2.5 功能测试

本系统按照完整数据库生命周期的操作流，设计了涵盖 DDL、DML 和 DQL 的综合测试用例。测试过程中，系统支持单条语句执行以及多条语句的连续批处理执行。具体功能测试结果如下：

测试 1：数据库级别的定义与操作 (Database DDL)

测试目的： 验证数据库的创建、防重校验、显示、删除及环境切换功能。

- **输入：**

CREATE DATABASE XJGL;

CREATE DATABASE JUST_FOR_TEST;

```
CREATE DATABASE JUST_FOR_TEST;
```

```
SHOW DATABASES;
```

```
DROP DATABASE JUST_FOR_TEST;
```

```
SHOW DATABASES;
```

```
USE XJGL;
```

- 输出:

[执行成功] 数据库 'XJGL' 创建完毕!

[执行成功] 数据库 'JUST_FOR_TEST' 创建完毕!

[报错] 数据库 'JUST_FOR_TEST' 已存在!

Databases

XJGL

JUST_FOR_TEST

[执行成功] 数据库 'JUST_FOR_TEST' 已彻底删除。

Databases

XJGL

[执行成功] 当前数据库已切换至 'XJGL'。

测试 2：基本表级别的定义与操作 (Table DDL)

测试目的： 验证多表的创建、数据字典的隔离维护、表的显示与删除。

- **输入：**

```
CREATE TABLE STUDENT(SNAME CHAR(20),SAGE INT,SSEX INT);
```

```
CREATE TABLE COURSE(CNAME CHAR(20),CID INT);
```

```
CREATE TABLE CS(SNAME CHAR(20),CID INT);
```

```
CREATE TABLE TEST_TABLE(COL1 CHAR(22),COL2 INT,COL3  
CHAR(22));
```

```
CREATE TABLE TEST_TABLE(COL1 CHAR(22),COL2 INT,COL3  
CHAR(22));
```

```
SHOW TABLES;
```

```
DROP TABLE TEST_TABLE;
```

```
SHOW TABLES;
```

- **输出：**

[执行成功] 表 'STUDENT' 创建完毕！

[执行成功] 表 'COURSE' 创建完毕！

[执行成功] 表 'CS' 创建完毕！

[执行成功] 表 'TEST_TABLE' 创建完毕！

[报错] 表 'TEST_TABLE' 已经存在！

Tables in XJGL

COURSE

CS

STUDENT

TEST_TABLE

[执行成功] 表 'TEST_TABLE' 已删除。

Tables in XJGL

COURSE

CS

STUDENT

测试 3：记录的插入与残缺列处理 (DML Insert)

测试目的： 验证全量字段插入以及指定列插入（验证底层系统是否能自动将未指定的列补全为 NULL）。

- **输入：**

```
INSERT      INTO      STUDENT(SNAME,SAGE,SSEX)      VALUES
('ZHANGSAN',22,1);
```

```
INSERT INTO STUDENT VALUES ('LISI',23,0);
```

```
INSERT INTO STUDENT(SNAME,SAGE) VALUES ('WANGWU',21);
```

```
INSERT INTO STUDENT VALUES ('ZHAOLIU',22,1);
```

```
INSERT INTO STUDENT VALUES ('XIAOBAI',23,0);
```

```
INSERT INTO STUDENT VALUES ('XIAOHEI',19,0);
```

```
INSERT INTO COURSE(CNAME,CID) VALUES ('DB',1);
```

```
INSERT INTO COURSE (CNAME,CID) VALUES('COMPILER',2);
```

```
INSERT INTO COURSE (CNAME,CID) VALUES('C',3);
```

- 输出:

[执行成功] 插入记录 -> ZHANGSAN 22 1

[执行成功] 插入记录 -> LISI 23 0

[执行成功] 插入记录 -> WANGWU 21 NULL

[执行成功] 插入记录 -> ZHAOLIU 22 1

[执行成功] 插入记录 -> XIAOBAI 23 0

[执行成功] 插入记录 -> XIAOHEI 19 0

[执行成功] 插入记录 -> DB 1

[执行成功] 插入记录 -> COMPILER 2

[执行成功] 插入记录 -> C 3

测试 4: 单表复合条件查询 (DQL Single Table)

测试目的: 验证嵌套括号脱壳、比较运算符（包括!=）以及 AND/OR 抽象语法树逻辑的递归求值。

- 输入:

```
SELECT SNAME,SAGE FROM STUDENT WHERE (((SAGE=21)));
```

```
SELECT SNAME,SAGE FROM STUDENT WHERE (SAGE>21) AND  
(SSEX=0);
```

```
SELECT * FROM STUDENT WHERE SSEX!=1;
```

- 输出:

SNAME	SAGE
-------	------

WANGWU	21
--------	----

共查询到 1 条记录。

SNAME	SAGE
-------	------

LISI	23
------	----

XIAOBAI	23
---------	----

共查询到 2 条记录。

SNAME	SAGE	SSEX
-------	------	------

LISI	23	0
------	----	---

WANGWU	21	NULL
--------	----	------

XIAOBAI	23	0
---------	----	---

XIAOHEI	19	0
---------	----	---

共查询到 4 条记录。

测试 5：多表联合查询、更新与删除 (Join, Update & Delete)

测试目的： 验证多表笛卡尔积组合机制，以及同时设定多个字段值的 UPDATE 语句的引擎执行能力。

- 输入：

-- 1. 带条件的多表联合查询

```
SELECT * FROM STUDENT,COURSE WHERE (SSEX=0) AND (CID=1);
```

-- 2. 复合条件删除

```
DELETE FROM STUDENT WHERE (SAGE>21) AND (SSEX=0);
```

-- 3. 多字段批量更新

```
UPDATE      STUDENT      SET      SAGE=27,SSEX=1      WHERE  
SNAME='ZHANGSAN';
```

```
SELECT * FROM STUDENT;
```

- 输出：

```
-----  
STU.SNAME   | STU.SAGE    | STU.SSEX    | COU.CNAME    |  
COU.CID
```

```
-----  
LISI         | 23          | 0           | DB           | 1  
XIAOBAI      | 23          | 0           | DB           | 1  
XIAOHEI      | 19          | 0           | DB           | 1  
-----
```

共查询到 3 条记录。

[执行成功] DELETE 删除了 2 行记录。

[执行成功] UPDATE 影响了 1 行记录。

SNAME	SAGE	SSEX

ZHANGSAN	27	1
WANGWU	21	NULL
ZHAOLIU	22	1
XIAOHEI	19	0

共查询到 4 条记录。

3 总结与未来工作

3.1 未成功能

1. **底层物理索引机制：**当前系统的数据过滤（如 `WHERE SAGE = 20`）强依赖于在内存中按行遍历读取整个 `.txt` 文件，属于低效的全表顺序扫描（`Sequential Scan`）。尚未实现 `B+` 树或 `Hash` 散列等高级物理索引结构。
2. **关系完整性约束：**在数据操纵层面，系统尚未实现对主码（`Primary Key`）唯一性的硬性检测，也未实现外码（`Foreign Key`）的参照完整性约束以及 `NOT NULL` 等实体完整性约束。
3. **事务管理与并发控制：**系统尚未引入事务的 `ACID` 特性机制。当前的 `UPDATE` 和 `DELETE` 采用的是纯文本全量覆写模式，缺乏读写锁（`S` 锁/`X` 锁）以及预写式重做日志（`Redo Log`）的支持，中途崩溃极易导致数据丢失，无法支持高并发访问。
4. **视图与安全权限：**目前的元数据（`sys.dat`）仅记录了数据库和表的基础 `Schema`，尚未扩展用户角色表（`User/Role Catalog`），不支持基于用户的权限鉴权（`GRANT/REVOKE`），也未实现基于查询结果集的虚拟视图（`View`）功能。
5. **查询优化器：**在多表联合查询（`SELECT * FROM A, B`）时，系统目前采用的是暴力的全量笛卡尔积（`Cartesian Product`）生成虚拟宽表后再进行 `check_condition` 过滤。未实现启发式或基于代价的逻辑查询优化（如将选择操作尽可能下推至叶子节点）。

3.2 未来实现方案

1. 基于 B+ 树的索引层实现方案

废弃单纯的文本按行存储机制，引入“页（Page）”作为最小磁盘 I/O 单位（如固定 4KB 一页）。在存储引擎之上构建缓冲池管理器（Buffer Pool Manager），并在内存中基于 Python 面向对象特性实现 B+ 树结构。建表时指定主键，引擎自动为其构建聚集索引；通过索引遍历定位数据的物理块偏移量（Offset），将点查的时间复杂度从 $O(N)$ 降至 $O(\log N)$ 。

2. 关系完整性约束的实现方案

扩展词法与语法分析器，使 AST 支持 PRIMARY KEY 标记。在元数据文件 sys.dat 中增加一列记录约束类型。在执行 execute_insert 与 execute_update 语义动作前，增加一重“完整性校验拦截器（Constraint Checker）”，利用 B+ 树快速查重，若破坏实体完整性则直接抛出异常并回滚。

3. 基于 WAL (Write-Ahead Logging) 的事务与恢复方案

引入日志管理模块。任何 DML 操作在对数据页进行修改前，必须先生成一条包含旧值（Undo）和新值（Redo）的日志记录，并强制 flush 刷入本地的 .log 物理文件。若在执行文件覆写时程序崩溃，重启 DBMS 时引擎优先读取日志文件，利用 Redo 日志重放未落盘的提交，利用 Undo 日志撤销未提交的脏数据，从而保障原子性与持久性。

4. 查询优化器的逻辑重写方案

在语法分析器生成 AST 树与底层的 execute_select 执行器之间，插入一个查询优化层（Query Optimizer）。利用关系代数等价变换规则（RBO），重构 AST 的节点顺序：

- 。 **选择下推 (Push-down Selection):** 将 WHERE 树中的条件节点尽可能推向叶子表的上方，在做表连接前先过滤掉不相关的元组。

- 。 **联查优化:** 将带有等值条件的 CROSS JOIN (笛卡尔积) 转化为 INNER JOIN (内连接, 如采用 Hash Join 算法), 从而彻底解决多表联查产生的庞大内存消耗瓶颈。